



BEAT BATTLE ROYALE

Developer Specification · Screens & Modi

Dieses Dokument beschreibt Lobby-Aufbau, Spielmodi-Abläufe und technische Anforderungen. Kein Punktesystem – Fokus auf Screen-Flow und Implementierungslogik.

Version	1.0 (Spec-Dokument ohne Scoring)
Zielgruppe	Entwickler – Frontend & Backend
Spielerzahl	1–6 Personen (lokal oder remote via Voicechat)
Neue Modi	7 Spielmodi gesamt (inkl. 2 neuer Modi in diesem Dokument)

1. LOBBY & ANMELDEMASKE

1.1 Audio-Test (vor allem anderen)

Bevor irgendein Setup-Screen erscheint, wird ein kurzes Audio-Signal abgespielt. Erst wenn der Nutzer bestätigt, dass er den Ton gehört hat, geht es weiter.

DEV-HINWEIS: Button „Ton gehört ✓“ muss aktiv geklickt werden – kein Auto-Skip. Falls kein Ton: Link zu Browser-Audioeinstellungen anzeigen.

1.2 Spieler-Anzahl & Namen

- 1 Auswahl der Spieleranzahl: 1 – 6 Personen (Stepper oder Buttons).
- 2 Für jeden Spieler: Eingabefeld für den Namen (Pflichtfeld, max. 20 Zeichen).
- 3 Namen erscheinen später im Buzzer-Tracker und Scoreboard.
- 4 Multiplayer-Hinweis einblenden: „Remote-Modus: Alle Spieler öffnen die App auf ihrem Gerät und kommunizieren per Voicechat (z.B. Discord).“

DEV-HINWEIS: Namen müssen session-weit als Array gespeichert sein. Bei Remote-Modus: kein Echtzeit-Sync erforderlich – jeder Spieler sieht seine eigene App-Instanz.

1.3 Jahrzehnt-Auswahl

Mehrfachauswahl möglich. Mindestens ein Jahrzehnt muss gewählt sein.

Auswahl	Zeitraum	Typische Genres (Beispiele – in DB hinterlegen!)
60er	1960–1969	Rock 'n' Roll, Beat, Soul, Folk, Psychedelic Rock
70er	1970–1979	Disco, Funk, Soul, Classic Rock, Prog Rock, Punk
80er	1980–1989	Pop, Synthpop, New Wave, Hair Metal, Hip-Hop (früh), Schlager
90er	1990–1999	Pop, Grunge, R&B, Eurodance, Hip-Hop, Britpop, Techno
2000er	2000–2009	Pop, R&B, Hip-Hop, Emo, Rock, Electronic, Schlager
2010er	2010–2019	Pop, EDM, Trap, Indie Pop, K-Pop, Latin Pop
Aktuell	2020–heute	Pop, Afrobeats, Hyperpop, Latin, Hip-Hop, Electronic

DEV-HINWEIS: In der Datenbank muss pro Jahrzehnt eine Liste typischer Genres hinterlegt sein. Wenn der Nutzer ein Jahrzehnt auswählt, werden im nächsten Screen (Genre-Auswahl) automatisch nur die für dieses Jahrzehnt relevanten Genres angezeigt. Bei Mehrfachauswahl von Jahrzehnten: UNION der Genre-Listen bilden (Duplikate entfernen). Genres sind NICHT global fest – sie sind jahrzehntabhängig und kommen aus der DB.

1.4 Genre-Auswahl

Die angezeigten Genres werden dynamisch aus der Jahrzehnt-Selektion (1.3) generiert. Mehrfachauswahl möglich. Mindestens ein Genre muss aktiv sein.

Beispiel-Genres (vollständige Liste kommt aus DB):

Pop	Rock	Hip-Hop / Rap	R&B; / Soul	Electronic / Dance
Schlager / Deutsch	Metal / Hard Rock	Indie / Alternative	Latin	Disco / Funk
Grunge	Techno / House	Eurodance	K-Pop	Country / Folk

DEV-HINWEIS: Genre-Filter bestimmt, welche Songs aus dem YouTube-Pool gezogen werden. Song-Suche muss Genre-Tag-Matching unterstützen.

1.5 Modus-Auswahl

Nach Genre-Auswahl wählen die Spieler einen der 7 Spielmodi. Jeder Modus wird als Karte mit Kurzbeschreibung angezeigt.

■ Classic Mode	Song von Anfang an, normale Geschwindigkeit
■ Random Start	Zufälliger Einstieg zwischen Sek. 20–55
■ Wissens-Modus	5 Fakten → Biet-Runde → Musik-Versuch
■ Speed Ramp	Song startet langsam, wird stufenweise schneller
■ Wett-Modus	Vorab-Wette: in wie vielen Sekunden erkenne ich den Song?
■ Volume Fade-In	Video sichtbar, Ton startet stumm und wird langsam lauter
■ Dual Stream	Video stumm + separater Audio-Track gleichzeitig

DEV-HINWEIS: Classic und Random Start: Rundenanzahl wählbar (5 / 10 / 15 / 25). Alle anderen Modi: Rundenanzahl fix (5 Runden).

2. SPIELMODI – ABLAUF & TECHNISCHE ANFORDERUNGEN

■ MODUS 1 — CLASSIC MODE

Startpunkt	Sekunde 0 (Anfang des Songs)
Geschwindigkeit	1× (normal, unveränderlich)
Buzzer	Maus-Klick oder Leertaste
Rundenanzahl	Wählbar: 5 / 10 / 15 / 25

1	Song wird ab Sekunde 0 in normaler Geschwindigkeit gestartet.
2	Jeder Spieler kann jederzeit den Buzzer drücken (Klick oder Leertaste).
3	Lied stoppt sofort. 5-Sekunden-Countdown läuft rückwärts. Bei 0: Audio-Signal (Beep).
4	Spieler nennt Antwort laut. Punkte-Tracker erscheint auf dem Screen.
5	Im Tracker: Spielernamen auswählen + Ergebnis eintragen (Richtig / Falsch / Nichts gesagt).
6	Auflösung einblenden: Titel, Interpret, (Cover-Bild falls verfügbar).
7	Nächste Runde oder Spiel-Ende-Screen.

■ MODUS 2 — RANDOM START

Startpunkt	Zufällig: Sekunde 20 – 55
Geschwindigkeit	1× (normal)
Buzzer	Maus-Klick oder Leertaste
Rundenanzahl	Wählbar: 5 / 10 / 15 / 25

1	System zieht zufälligen Startpunkt zwischen Sek. 20 und Sek. 55.
2	Song startet an diesem Punkt, keine Anzeige welche Sekunde.
3	Buzzer-Mechanismus identisch zu Classic Mode.
4	Countdown, Punkte-Tracker und Auflösung identisch zu Classic Mode.

DEV-HINWEIS: Startpunkt per `Math.random()` o.ä. generieren. YouTube IFrame API: `ytPlayer.seekTo(startSek)` vor `playVideo()`.

■ MODUS 3 — WISSENS-MODUS

Phase 1	5 Fakten, jeweils 5 Sek. Abstand, bleiben alle sichtbar
Phase 2	Biet-Runde: Spieler wählen ihre Sekunden-Wette
Phase 3	Musik läuft exakt so lange wie geboten, dann Stop
Rundenanzahl	Fix: 5 Runden

Phase 1 – Fakten-Einblendung:

1	Fakten werden einzeln eingeblendet, Reihenfolge: sehr schwer → sehr leicht.
2	Jeder Fakt: max. 1 Satz. Zeitversatz: 5 Sekunden zwischen den Fakten.
3	Bereits angezeigte Fakten bleiben untereinander sichtbar (kein Ausblenden).
4	Nach Fakt 5: kurze Pause (1–2 Sek.), dann automatisch Phase 2.

Phase 2 – Biet-Runde:

1	Alle Spieler sehen die 5 Fakten und diskutieren (per Voicechat bei Remote).
2	Jeder Spieler kann seinen Namen wählen und eine Zeit-Option antippen.
3	Verfügbare Zeiten: 0,5s / 1s / 2s / 3s / 4s / 5s / 6s / 8s / 10s / 15s.
4	Spieler mit der niedrigsten Zeit erhält den Versuch. Bei Gleichstand: Buzzer-Race.

Phase 3 – Musik-Versuch:

1	Song startet ab Sekunde 0, läuft exakt so viele Sekunden wie geboten.
2	Nach Ablauf: Musik stoppt automatisch. Spieler nennt Antwort.
3	Punkte-Tracker und Auflösung identisch zu Classic Mode.

DEV-HINWEIS: KI-generierte Fakten via API (z.B. Anthropic/OpenAI) abrufen. Prompt: "Gib 5 Fakten über [Song] von sehr schwer bis leicht, je max. 1 Satz, JSON-Array." Fakten cachen um API-Calls zu minimieren.

■ MODUS 4 — SPEED RAMP

Startpunkt	Sekunde 0
Geschwindigkeit	Stufenweise steigend: 0,25× → 2×
Buzzer	In jeder Phase möglich (Klick oder Leertaste)

Rundenanzahl	Fix: 5 Runden
--------------	---------------

Phase	Aktion / Zustand	Dauer	Anmerkung
Phase 1	0,25× – sehr langsam	2 Sek.	Kaum erkennbar
Phase 2	0,50× – halb	2 Sek.	–
Phase 3	0,75× – fast normal	2 Sek.	–
Phase 4	1,00× – normal	2 Sek.	Gut erkennbar
Phase 5	1,25× – etwas schnell	2 Sek.	–
Phase 6	1,50× – schnell	5 Sek.	–
Phase 7	2,00× – doppelt	10 Sek.	Letzter Versuch
Ende	Song stoppt automatisch	–	0 Punkte falls kein Buzzer

DEV-HINWEIS: YouTube IFrame API: `setPlaybackRate()` für Stufen 0.25 / 0.5 / 0.75 / 1 / 1.25 / 1.5 / 2. Phasen-Timer per `setTimeout()` steuern. Buzzer in jeder Phase abfangen und aktuelle Phase für Punkte-Logik speichern.

■ MODUS 5 — WETT-MODUS

Startpunkt	Sekunde 0, normale Geschwindigkeit
Prinzip	Spieler wettet vorab: in X Sekunden werde ich buzzern
Wett-Zeiten	3s / 5s / 10s / 15s / 20s / 25s / 30s
Rundenanzahl	Fix: 5 Runden

1	Vor Songstart: Jeder Spieler wählt seinen Namen und eine Wett-Zeit.
2	Song startet. Pro Spieler läuft ein individueller Timer entsprechend seiner Wette.
3	Spieler muss innerhalb seiner Wett-Zeit buzzern UND richtig antworten.
4	Nach Ablauf der Wett-Zeit: Spieler hat verloren (0 Punkte für diese Runde).
5	Punkte-Tracker und Auflösung identisch zu Classic Mode.

DEV-HINWEIS: Pro Spieler eigenen Countdown-Timer führen. Alle Timer gleichzeitig beim Song-Start starten. Timer-Ablauf markiert den Spieler als "verfallen" – kein Buzzer mehr möglich.

■ MODUS 6 — VOLUME FADE-IN

Video	Sichtbar (normales YouTube-Video)
Audio	Startet stumm (0%), steigt alle 2 Sek. um 5%
Max. Lautstärke	50% (nach 20 Sekunden erreicht)
Max. Laufdauer	20 Sekunden
Buzzer	Identisch zu Classic Mode (Klick oder Leertaste)
Rundenanzahl	Fix: 5 Runden

Ablauf:

1	Song startet ab Sekunde 0. Video ist vollständig sichtbar.
2	Lautstärke startet bei 0% (komplett stumm). Kein Ton für die ersten 2 Sekunden.
3	Alle 2 Sekunden erhöht sich die Lautstärke um 5%.
4	Nach 20 Sekunden: Maximallautstärke von 50% erreicht. Song stoppt automatisch.
5	Spieler können in jeder Phase den Buzzer drücken.
6	Buzzer → Song stoppt → 5-Sek.-Countdown → Antwort → Punkte-Tracker → Auflösung.

Lautstärke-Verlauf:

Zeitfenster	Lautstärke	Hinweis
0–2s	0% (stumm)	–
2–4s	5%	–
4–6s	10%	–
6–8s	15%	–
8–10s	20%	–
10–12s	25%	–
12–14s	30%	–
14–16s	35%	–
16–18s	40%	–
18–20s	45%	Letzter Schritt vor Auto-Stop
20s	50% → STOP	Song endet automatisch

DEV-HINWEIS: YouTube IFrame API: `ytPlayer.setVolume(n)` setzt Lautstärke 0–100. Alle 2 Sekunden `setInterval()` aufrufen und Lautstärke um 5 erhöhen. Bei `Volume >= 50` oder `elapsed >= 20s`: `clearInterval()` + `stopVideo()`. Video-Element bleibt sichtbar (kein Cover/Overlay). Achtung: `setVolume()` ist unabhängig von `mute` – sicherstellen, dass Player NICHT gemutet ist.

■ MODUS 7 — DUAL STREAM

Stream A	YouTube-Video – stumm (Muted, kein Ton)
Stream B	Separater YouTube-Song – nur Audio (Video versteckt)
Ziel	Spieler müssen BEIDE Songs erkennen
Laufdauer	Beide Streams laufen 30 Sekunden gleichzeitig
Buzzer	Identisch zu Classic Mode
Rundenanzahl	Fix: 5 Runden (pro Runde = 1 Videopaar)

Ablauf:

1	System zieht zwei unterschiedliche Songs aus dem Pool (selbes Jahrzehnt/Genre oder gemischt – konfigurierbar).
2	Stream A: YouTube-Video wird angezeigt, Lautstärke = 0 (stumm). Spieler sehen das Video.
3	Stream B: Zweiter YouTube-Player läuft versteckt (off-screen / hinter Cover), Lautstärke normal.
4	Beide Streams starten gleichzeitig und laufen 30 Sekunden.
5	Nach 30 Sekunden: beide Streams stoppen automatisch.
6	Spieler können auch vor Ablauf der 30 Sek. den Buzzer drücken.
7	Buzzer → beide Streams stoppen → 5-Sek.-Countdown → Spieler nennt BEIDE Songs.
8	Punkte-Tracker erscheint mit zwei Eingabefeldern: „Song A (Video)“ und „Song B (Audio)“.
9	Auflösung: Beide Titel/Interpreten werden eingeblendet.

Layout-Anforderung:

Bereich	Inhalt	Sichtbar?
Obere Hälfte	Stream A: YouTube-Video (stumm)	✓ Ja – Video sichtbar, kein Ton
Untere Hälfte	Stream B: Audio-Only Cover (Equalizer-Animation o.ä.)	Video versteckt, nur Ton

DEV-HINWEIS: Zwei separate YouTube IFrame API Instanzen initialisieren (ytPlayerA, ytPlayerB). Player A: `setVolume(0)` oder `mute()` – Video bleibt sichtbar. Player B: off-screen positionieren (`position:fixed; top:-9999px`) – nur Audio. Beide per `playVideo()` gleichzeitig starten. Timer nach 30s: beide `stopVideo()`. Punkte-Tracker braucht 2 Antwortfelder + 2 Auflösungs-Slots.

3. SCREEN-FLOW ÜBERSICHT

Screen	Inhalt / Aktion	Weiter zu
① Audio-Test	Ton-Test + Bestätigung durch Spieler	②
② Spieler-Setup	Anzahl (1–6) + Namen eingeben	③
③ Jahrzehnt	Mehrfachauswahl Jahrzehnte → triggert Genre-DB-Query	④
④ Genre	Gefilterte Genre-Liste (aus DB), Mehrfachauswahl	⑤
⑤ Modus	Auswahl aus 7 Modi + Rundenanzahl (wo wählbar)	⑥
⑥ Spiel-Loop	Modus-spezifischer Ablauf (s. Abschnitt 2)	⑥ (nächste Runde)
⑦ Ende-Screen	Finales Scoreboard, Gewinner, Restart-Option	② (Neustart)

4. OFFENE ENTWICKLUNGS-PUNKTE

Datenbank Jahrzehnt → Genre Mapping

Pro Jahrzehnt muss eine Genre-Liste in der DB hinterlegt sein. Diese wird bei Jahrzehnt-Auswahl live abgefragt.

KI-Fakten im Wissens-Modus

Fakten per API generieren oder vorab für alle Songs cachen. Fallback wenn API nicht verfügbar.

Dual YouTube-Instanzen

Zwei simultane YT-Player auf einer Seite – Autoplay-Policy beachten (Browser blockiert zweiten Autoplay).

Volume Fade-In Lautstärke

setVolume() vs. mute() – Unterschied beachten. Player darf nicht gemutet sein, sonst greift setVolume() nicht.

Lizenz-Fehler-Handling

Error 150/101: automatischer Skip. Nächsten Song aus Pool laden ohne Spielunterbrechung.

Remote Multiplayer

Kein Echtzeit-Sync nötig. Spieler nutzen gleiche App-URL. Kommunikation per externem Voicechat. Scoreboard wird manuell eingegeben.

Buzzer-Race bei Gleichstand

Im Wissens-Modus bei gleicher Biet-Zeit: wer zuerst buzzert gewinnt den Versuch. Server-seitiger Timestamp empfohlen.

Beat Battle Royale · Developer Spec v1.0 · Ohne Scoring-Details

Punktesystem und detaillierte Scoring-Tabellen: siehe Dokument v2.0